... which offers `::add()` and `::remove()` methods to make manipulating classes much easier:

```
$lastPara->classList->remove('small');
$lastPara->classList->add('large');
$firstPara->classList->remove('large');
$firstPara->classList->add('small');

print $dom->saveHTML();

Output:
...
<main class="main-content">
  <p class="large">Para Three</p><p class="small">Para One</p>
```

`Dom\TokenList::remove()` will accept a variable number of string arguments to allow the removal of multiple classes in one call. If an element had `class="small green bold article"` classes, then `classList->remove('small', 'green', 'bold');` would result in the element retaining just `class="article"`. The `TokenList` class also offers `::contains(token)` to search the list for a given token and `::replace(token, newToken)` to swap one token for another in a single call. The previous example could have also been written as:

```
$lastPara->classList->replace('small', 'large');
$firstPara->classList->replace('large', 'small');
```

Unfortunately, PHP <= 8.3 does not offer an equivalent interface for class manipulation; there is no `DOMTokenList` class I'm afraid. You'll be forced to jump through string handling hoops to make changes, but `DOMElement` does expose the classes in a `className` string property.

Manipulating Other Attributes

Of course, other attributes will need attention too. The public `Dom\Element::id` string property can be directly read/written to control any element's `id` property (no need for fancy lists and utility methods; the `id` attribute is only ever a single value – just make sure they're unique throughout the DOM). All other attributes can be accessed via the `Dom\Node::attributes` property...